IMCRYPTO: An In-Memory Computing Fabric for AES Encryption and Decryption

Dayane Reis, *Member, IEEE*, Haoran Geng, Michael Niemier, *Senior Member, IEEE*, and Xiaobo Sharon Hu, *Fellow, IEEE*

***Abstract*— This article proposes IMCRYPTO, an in-memory computing (IMC) fabric for accelerating advanced encryption standard (AES) encryption and decryption. IMCRYPTO employs a unified structure to implement encryption and decryption in a single-hardware architecture with combined (Inv)SubBytes and (Inv)MixColumns steps. Because of this step combination and the high parallelism achieved by multiple units of random access memory (RAM) and random access/content addressable memory (RA/CAM) arrays, IMCRYPTO achieves high-throughput encryption and decryption without sacrificing area and power consumption. In addition, due to the integration of an RISC-V core, IMCRYPTO offers programmability and flexibility. IMCRYPTO improves the throughput per area by a minimum (maximum) of 3.3× (223.1×) compared to previous ASICs/IMC architectures for AES-128 encryption. Projections show added benefit from emerging technologies of up to 5.3× to the area–delay–power product of IMCRYPTO.**

***Index Terms*— Advanced encryption standard (AES), content addressable memory (CAM), in-memory computing (IMC), random access memory (RAM).**

I. Introduction

ADVANCED encryption standard (AES) [1] is a widely used encryption method used in a variety of computer systems. Despite the excellent security of AES, its software-based implementations can be computationally expensive, resulting in low throughput. High-throughput AES encryption/decryption is extremely desirable in many applications (e.g., virtual private networks and electronic financial transactions). As demonstrated in previous work (e.g., [2]–[7]), hardware accelerators designed for AES encryption/decryption can achieve high throughput while enabling area-efficient design alternatives for resource-constrained environments, such as edge computing. AES hardware accelerators are usually application-specific integrated circuits (ASICs) or field-programmable gate array (FPGA)-based coprocessors that implement the steps for the AES algorithm, i.e., AddRoundKey, SubBytes/InvSubBytes, ShiftRows, and MixColumns/InvMixColumns.

Manuscript received October 8, 2021; revised February 13, 2022; accepted March 1, 2022. Date of publication March 17, 2022; date of current version April 26, 2022. This work was supported in part by ASCENT, one of six centers in JUMP, through a Semiconductor Research Corporation (SRC) Program sponsored by Defense Advanced Research Projects Agency (DARPA). *(Corresponding author: Xiaobo Sharon Hu.)*

The authors are with the Department of Computer Science and Engineering, University of Notre Dame, Notre Dame, IN 46556 USA (e-mail: shu@nd.edu).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TVLSI.2022.3157270.

Digital Object Identifier 10.1109/TVLSI.2022.3157270

The core functions in AES involve mainly simple operations, which makes AES encryption/decryption memory bound. To address the memory bottleneck of AES, in-memory computing (IMC) architectures (i.e., with modified memory cells or peripherals) may be designed to perform operations commonly found in AES-based encryption/decryption (e.g., bit shifts, byte permutations, and XOR). IMC-based architectures operate on memory words without the need for data transfers to a processing unit (e.g., [6] and [7]). Despite the potential for reduced data traffic, work in [6] and [7] must make tradeoffs between area/power efficiency and parallelism. To this end, accelerators based on the lookup table (LUT) and precomputed operations (e.g., [2] and [3]) have demonstrated high levels of parallelism and throughput. In this regard, memories based on emerging technologies (e.g., phase-change memories (PCMs), resistive random access memories (RRAMs), spin-transfer torque magnetoresistive random access memories (STT-MRAMs), and ferroelectric field-effect transistor-based random access memories (FeFET-RAMs)] can considerably improve the density and static power consumption of IMC architectures (albeit at the expense of longer write latencies) [8]–[10].

Besides architecture and technology considerations, security and flexibility in supporting different modes of AES are desirable features of hardware accelerators. For the former, the ability to run new encryption/decryption algorithms that resist emerging attacks is highly desirable [11]. For the latter, different modes of AES may be selected for different applications, e.g., cipher block chaining (CBC), and the counter (CTR) mode can be used for data streaming and for protecting data storage devices [12]–[14]. As another example, authenticated encryption guarantees both confidentiality (data secrecy) and integrity (data authenticity), which has utility in distinct application spaces and can be performed with Galois/counter mode (GCM). Considering this wide range of usage scenarios, having an AES accelerator that supports different modes of AES may reduce costs with system/intellectual property (IP) redesign. As an additional benefit, a general-purpose programmable accelerator for AES could accommodate other symmetric block ciphers and hashing functions, such as the Secure Hash Algorithm (SHA)-256 or the Message Digest Algorithm (MD)-5.

In this article, we propose a programmable IMC fabric for AES acceleration (IMCRYPTO) that achieves the same parallelism as LUT methods while avoiding large area/power overheads. IMCRYPTO is based on both conventional and

1063-8210 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

compute-enabled random access memory (RAM) arrays [15], as well as dual-function random access/content addressable memory (RA/CAM) arrays. Specifically, the RA/CAM arrays employed in IMCRYPTO enable encryption and decryption to be performed with a common structure. To improve the computational efficiency, the steps SubByte/MixColumns (for encryption) and InvSubBytes/AddRoundKey/InvMixColumns (for decryption) are combined and executed in one step with our design. The proposed fabric supports pipelining and enables high-throughput execution. The contributions of our work are summarized as follows.

1) We propose IMCRYPTO, a highly parallel IMC fabric based on RAM and RA/CAM arrays that provide high-throughput AES encryption/decryption with a common hardware structure.
2) We propose a compact implementation of the AES algorithm with IMCRYPTO, which combines certain steps in encryption (and decryption) to reduce computation load.
3) We propose an RISC-V-based controller that implements customized instructions in IMCRYPTO. The flexibility of IMCRYPTO enables multiple modes of AES and other block ciphers to be implemented.

We have performed detailed circuit-level, simulation-based evaluations of our proposed IMCRYPTO architecture. We consider figures of merit, such as delay, area, power, throughput, area–delay–power product (ADPP), and throughput per area, and compare them with state-of-the-art ASIC and IMC-based accelerators for AES encryption [4]–[7]. Results indicate minimum (maximum) throughput per area improvements of 3.3× (223.1×) and minimum (maximum) ADPP improvements of 1.2 × (6238.3×) for CMOS-based IMCRYPTO compared to ASIC and IMC accelerators for AES-128 encryption of a 1-MB plaintext (PT). (The wide spectrum of improvement is a result of different circuit and design paradigms employed in each accelerator.) IMCRYPTO can also leverage emerging memory technologies to further boost the ADPP improvements over a conventional CMOS-based implementation. Our analysis suggests that employing emerging technologies in the design of memory units in IMCRYPTO could improve the ADPP by an additional factor of up to 5.3×.

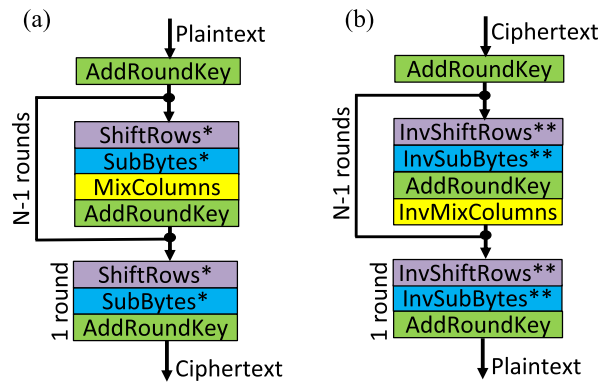
II. Background and Related Work

In this section, we briefly review the basics of AES encryption/decryption and discuss related work on hardware accelerators for AES encryption/decryption.

A. AES Basics

Fig. 1(a) and (b) illustrates the steps performed in N rounds of AES encryption and decryption. As a variant of the Rijndael block cipher, AES employs a fixed block size of 128 bits and a key size of 128, 192, or 256 bits (for AES-128, AES-192, or AES-256, respectively). Round keys are derived from the main key for each one of the N rounds using a key schedule algorithm [1].

The 128-bit block is arranged as a 4×4 array of bytes, which is the state array. Each step in the AES encryption (decryption)



*ShiftRows and SubBytes **(InvShiftRows and InvSubBytes) are commutative steps in AES encryption (decryption) [1].

Fig. 1. Steps performed in the N rounds of AES: (a) encryption and (b) decryption. The values of N are 10 (for AES-128), 12 (for AES-192), and 14 (for AES-256). (Inv)ShiftRows and (Inv)SubBytes are commutative steps [1].

takes a state array as an input and produces an output state array that is passed to the next step (or returned as the final encryption or decryption result at the end of N rounds). We represent elements of the input (output) state array as $a_{i,j}$ ($b_{i,j}$), where $i = 0, 1, 2, 3$ and $j = 0, 1, 2, 3$. The steps of AES encryption (decryption) are summarized in the following.

1) AddRoundKey: In the AddRoundKey step, every byte of a round key rk is combined to the corresponding byte of the input state through an XOR operation, i.e., $b_{i,j} = a_{i,j} \oplus rk_{i,j}$.

2) SubBytes and InvSubBytes: During SubBytes and InvSubBytes, each byte $a_{i,j}$ in the input state array is replaced by a substitution byte to form the output state array, i.e., $b_{i,j} = \text{Sbox}(a_{i,j})$. Since each byte $a_{i,j}$ in the 0×00–0xFF interval maps to a unique substitution byte, encryption/decryption *Sbox* tables can be precomputed and stored in LUTs (or memory units) to improve the speed of AES. While prestored table-based implementations of the SubBytes and InvSubBytes steps may be vulnerable to cache timing attacks in a traditional central processing unit (CPU) architecture, they are well-suited for FPGA- or ASIC-based accelerator designs [16].

3) ShiftRows and InvShiftRows: During ShiftRows and InvShiftRows, the bytes of the input state array are circularly shifted to the left (right) by a fixed offset amount (zero, one, two, or three positions) depending on the index i of the byte. The output state is then formed by the byte-shifted input state. For $i = 0$, no circular byte shifts are performed ($b_{i,j} = a_{i,j}$). For $i = 1$, 2, and 3, the offsets of the circular byte shifts are defined as 1, 2, and 3, respectively, with a left (right) direction for encryption (decryption).

4) MixColumns and InvMixColumns: During MixColumns and InvMixColumns, a linear transformation is applied to the input state matrix to form the output state matrix. This transformation consists of a matrix multiplication over the Galois Field (GF(2^8)) between a fixed matrix and the input state matrix (see Fig. 2). As with SubBytes (InvSubBytes), it is possible to speed up the MixColumns (InvMixColumns) steps by precomputing the products between every byte value in the range 0×00–0xFF and the multiplication coefficients.

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

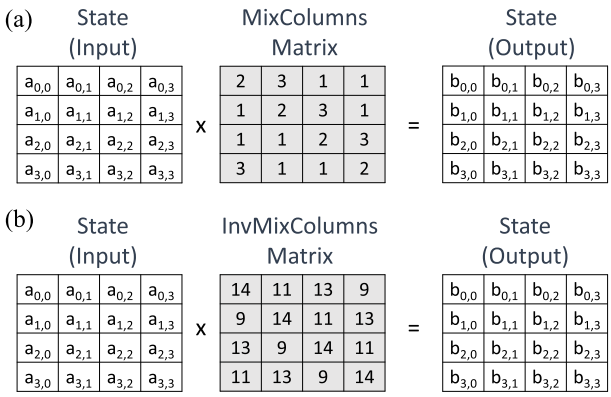


Fig. 2. (a) MixColumns and (b) InvMixColumns steps of AES-based encryption and decryption, respectively.

5) Modes of Operation: Modes of operation ensure confidentiality and authenticity when a cipher's single-block operation encrypts/decrypts an amount of data larger than a 128-bit block. Different modes of operation are commonly used in addition to the basic version of the AES algorithm a.k.a. the electronic code book (ECB). For instance, the CBC, the cipher feedback (CFB), the output feedback (OFB), the CTR, and the GCM are AES modes that require simple operations (such as XOR or addition) to be performed between the state array and randomly generated numbers (or counter values) at the start of every AES round. (For details on different modes of AES and their practical use, see [17] and [18]).

B. Related Work

Numerous research efforts have been devoted to developing FPGA- or ASIC-based hardware to accelerate AES (e.g., [2]–[7], [19], and [20]). That said, designing a hardware accelerator for AES that can provide high throughput without large overheads in terms of area and power consumption can be a challenging task. Area-efficient AES accelerators that provide high throughput are highly sought-after IP blocks for resource-constrained environments (e.g., in embedded systems [21]).

One of the strategies to minimize area without compromising performance is to reuse circuit blocks and design shared datapaths for encryption and decryption [4], [22]. In this regard, Ueno *et al.* [4] introduce an architecture for AES with shared encryption/decryption datapaths. Although their shared-datapath strategy can help reduce area, some circuits used for encryption and decryption still need to be implemented separately in this architecture. Furthermore, the design proposed in [4] is a standalone ASIC, which requires PTs and ciphertexts (CTs) to be fetched/stored in memory units. As we show in our evaluation (see Section VI), the latency and energy overheads from data transfers can significantly impact the performance of hardware accelerators for AES.

Reducing the impact of data transfers on a system's performance is a desired feature for AES accelerators. To this end, IMC could potentially alleviate the burden of data transfers between the memory and the accelerator. Zhang *et al.* [6] propose a reconfigurable cryptographic processor that uses IMC and near-memory computing for supporting AES and elliptic curve cryptography (ECC). Even though their IMC-based design improves the performance, it lacks the flexibility of supporting multiple modes of AES and other cryptographic algorithms (e.g., SHA-256). In this regard, Sayilar and Chiou [5] propose a highly reconfigurable ASIC cryptographic processor. It can support most of the existing cryptographic algorithms and potentially can support future algorithms. Although their throughput per area is much higher than the best CPU and graphics processing unit (GPU), they do not account for data movement in their evaluation (similar to [4]).

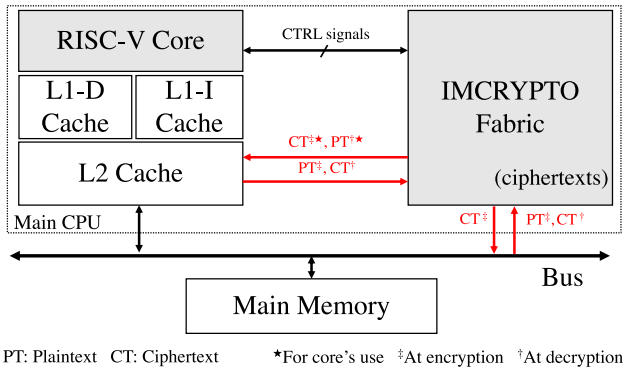


Fig. 3. Overview of an IMCRYPTO-based system.

Xie *et al.* [7] present another IMC accelerator for AES based on PCMs, which is intended to protect the contents of the nonvolatile, off-chip main memory in case an attacker gets physical access to it. The accelerator can simultaneously encrypt PT within each memory bank, and the entire encryption process can be completed without exposing the results to the memory bus. However, in a usage scenario where this IMC accelerator is used to perform decryption and send the resulting PT to the CPU, the architecture may be vulnerable to side-channel attacks as PT sent from the off-chip memory would be exposed to the memory bus. The design also requires expensive writes to the nonvolatile PCM (including results from the SubBytes and MixColumns steps of AES), which is a concern due to the high write latency and high write energy of the PCM memory [8].

III. IMCRYPTO Fabric

We propose the IMCRYPTO fabric for AES encryption/decryption. Our proposed in-memory accelerator is placed on the same chip as the CPU at the level of the last level cache (LLC). Placing IMCRYPTO on-chip and at the level of the LLC facilitates data exchange with the CPU and creates a mechanism for storing CTs inside IMCRYPTO's internal memory (i.e., IMCRYPTO works as the LLC for secure data). Furthermore, data are properly protected before being sent to the main memory through the memory bus. IMCRYPTO employs a lightweight, general-purpose core to program IMCRYPTO's functionality. For simplicity, we use an RISC-V core in IMCRYPTO, but other cores can also be designed for this purpose. Due to the programmability enabled by the RISC-V core, IMCRYPTO offers the needed flexibility at the application level (as mentioned in Section I).

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

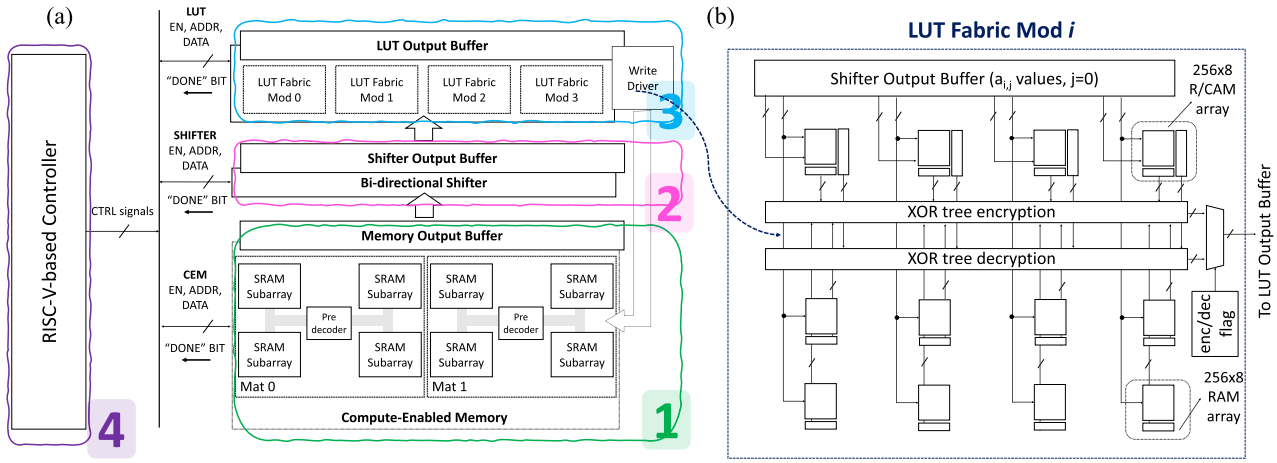


Fig. 4. (a) High-level view of the IMCRYPTO accelerator with (b) details of one LUT fabric module.

Note that using IMCRYPTO as the LLC for secure data should not significantly impact the performance of the main CPU for executing nonsecure workloads since the L1/L2 caches can have their content accessed directly by the RISC-V core without going through IMCRYPTO. Furthermore, with IMCRYPTO working simultaneously as an accelerator for AES encryption/decryption and the LLC for secure data, the L1 and L2 caches (represented in Fig. 3) are reserved for the storage of nonsensitive data. Thus, they can freely transfer data from/to the main memory through the bus and from/to the RISC-V core without any negative impact on the security.

Fig. 3 provides an overview of the IMCRYPTO-based system, where the interaction between IMCRYPTO and the other components of a computer system (i.e., the main memory and L1/L2 caches) is explicitly shown. Encryption/decryption requests to IMCRYPTO can be made by the RISC-V core, which communicates with IMCRYPTO through a set of control (CTRL) signals. The RISC-V uses the customized instruction "TEXT" (to be described in Section III-D and Table I) to pass the address of a PT or a CT for encryption/decryption with IMCRYPTO. Round keys are generated on the fly by the RISC-V core and directly provided to IMCRYPTO. In the following, we describe how IMCRYPTO processes encryption/decryption requests by the RISC-V core.

1) **At encryption**, the instruction "TEXT" is used to pass the data address of a PT to IMCRYPTO. The PT is fetched to (and *temporarily* stored in) the internal memory of IMCRYPTO from either the L2 cache (in case of a cache hit) or the main memory (in case of a cache miss). After the data transfer process, the PT is encrypted by the IMC infrastructure in IMCRYPTO, generating a CT. Once encryption finishes, the resulting CT replaces the initial PT in the internal memory of IMCRYPTO, where it is stored for future use by the RISC-V core (i.e., IMCRYPTO works as the LLC for secure data). Once the capacity of the IMCRYPTO's memory is reached (or if there are conflict misses), selected CTs are transferred to the main memory via the bus.
2) **At decryption**, the instruction "TEXT" is used to pass the data address of a CT to IMCRYPTO. The CT is fetched to the internal memory of IMCRYPTO from either the L2 cache (in case of a cache hit) or the main memory (in case of a cache miss). After the data transfer process, the CT is decrypted by the IMC infrastructure in IMCRYPTO, generating a PT. Once decryption finishes, the postdecryption PT is not kept in the internal memory of IMCRYPTO (which is reserved for secure data) but rather is transferred to the CPU's L2 cache to be used by processes running on the CPU.

The IMCRYPTO fabric [see Fig. 4(a)] consists of circuits that may be grouped into four distinct blocks. The blocks, numbered from 1 to 4 in Fig. 4(a), are: 1) the compute-enabled memory (CEM); 2) the bidirectional shifter; 3) the LUT fabric (comprised of four modules); and 4) the RISC-V-based controller. The circuits in each block and the communication between them are described in the following.

A. Compute-Enabled Memory

The AddRoundKey step of AES encryption (decryption) is the addition of a round key to a PT (CT) through a simple XOR operation. To enable the AddRoundKey step in AES encryption (and in the initial round of AES decryption), the proposed architecture implements the CEM block [block 1 in Fig. 4(a)]. The CEM of IMCRYPTO employs circuits similar to those in [15] and [23]—i.e., customized sense amplifiers (SAs)—which enables bitwise Boolean logic between two memory words stored in a memory array.

The CEM receives PT from the CPU L2 cache, performs logic operations on the AES state array, and stores the resulting CT after encryption. The encrypted data in IMCRYPTO can be directly read from the CEM by the CPU cache structure and sent to external parties using communication protocols, such as the peer-to-peer (P2P) or attribute-based encryption (ABE) (used in IoT systems). Another usage scenario for direct access to the CEM is when the CPU core performs operations on data that have been previously encrypted and stored in IMCRYPTO. In this case, IMCRYPTO allows for

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

the data to be decrypted prior to sending it to be used by the processes running on the CPU core.

As an on-chip memory, the size of a CEM needs to be within the acceptable limits for a CPU cache (i.e., from tens of KB up to a few MB). A larger CEM enables more CTs to be stored in IMCRYPTO, which can be useful when cached CTs are sent to external parties using communication protocols. On the other hand, bigger memories have longer access times and consume more power. To allow for a compromise between memory size, access times, and energy consumption, the CEM block of IMCRYPTO is a 1-MB memory that consists of a tiled, compute-enabled SRAM structure [15]. With the aid of customized SAs, the CEM can execute XOR operations between two memory words without the need for reading the data out to an external processing unit.

B. Bidirectional Shifter Block

The bidirectional shifter block [labeled as block 2 in Fig. 4(a)] performs the byte permutations needed by the ShiftRows (InvShiftRows) steps of AES encryption (decryption). Internally, the circuit of the bidirectional shifter block is similar to that described in [20].

C. LUT Fabric

The LUT fabric [labeled as block 3 in Fig. 4(a)] is a critical block in IMCRYPTO as it executes the SubBytes/InvSubBytes and MixColumns/InvMixColumns (the most time-consuming steps of AES). With the use of small memory elements (i.e., 6T-SRAM and RA/CAM arrays of size 256×8) and customized peripherals, it is possible to perform these steps in a highly parallel and combined fashion with low latency. The size of the memory elements (i.e., 256×8) is chosen, so it is possible to store all the possible 256 precomputed bytes of combined SubBytes and MixColumns.

The LUT fabric block comprises four LUT fabric modules that execute one matrix multiplication in GF(2^8). These four identical LUT fabric modules in the LUT fabric block operate *in parallel* to compute each column of the output state array (i.e., $j = 0, 1, 2, 3$) in the MixColumns/InvMixColumns and SubBytes/InvSubBytes steps. The internal structure of an LUT fabric module is depicted in Fig. 4(b). Each LUT fabric module consists of: 1) four RA/CAM arrays with customized peripherals; 2) eight RAM arrays; and 3) two sets of cascaded XOR gates [a.k.a. XOR trees that are employed to perform additions over $GF(2^8)$]. The number of memory elements of each type inside an LUT fabric module allows combined steps to be performed without serializing operations. More details about the execution of combined steps will be given in Section IV-D.

D. RISC-V-Based Controller

Encryption and decryption in AES require that certain computational steps are performed in a certain order over N rounds (see Fig. 1). Moreover, different modes of AES have different data flows and may require additional operations. IMCRYPTO employs an RISC-V-based controller [block 4 in Fig. 4(a)] to perform the different steps of AES and support different modes of operation. In addition, the RISC-V-based controller makes it possible to extend the use of the IMCRYPTO fabric for other cryptographic algorithms that require similar operations as AES.

TABLE I

RV32I R-TYPE INSTRUCTIONS FOR IMCRYPTO. (a) RV32I I-TYPE INSTRUCTIONS FOR IMCRYPTO. (b) RV32I R-TYPE INSTRUCTIONS FOR IMCRYPTO

Instruction		Function Code		Assembly	Description
		Bits 1-7	Bit 8		
TEXT	L	0000000	0	TEXT address, $s1$	Load 128 bits from IMCRYPTO to RISC-V registers during decryption
	S		1	TEXT $s1$, address	Store 128 bits to IMCRYPTO from RISC-V registers during encryption
SFTR	E	0000100	0	SFTR $s1$	Trigger ShiftRows on $s1$ (encryption)
	D		1	SFTR $s1$	Trigger InvShiftRows on $s1$ (decryption)
SUBMX	E	0001000	0	SUBMX $s1$	Trigger combined SubBytes+ MixColumns (encryption) on $s1$
	D		1	SUBMX $s1$	Trigger combined InvSubBytes+ InvMixColumns (decryption) on $s1$
SBOX	E	0010000	0	SBOX $s1$	Trigger SubBytes (last round of encryption) on $s1$
	D		1	SBOX $s1$	Trigger InvSubBytes (last round of decryption) on $s1$

(a)

Instruction	Function Code		Assembly	Description
	Bits 1-7	Bits 8-10		
IMMOVE	0000000	000	IMMOVE $s1$, sd	Move data in memory from address $s1$ to sd
IMADD	0000000	001	IMADD $s1$, $s2$, sd	Store the value of $s1 + s2$ into sd
IMAND	0000000	010	IMAND $s1$, $s2$, sd	Store the value of $s1$ AND $s2$ into sd
IMOR	0000000	011	IMOR $s1$, $s2$, sd	Store the value of $s1$ OR $s2$ into sd
IMXOR	0000000	100	IMXOR $s1$, $s2$, sd	Store the value of $s1$ XOR $s2$ into sd
IMNOT	0000000	101	IMNOT $s1$, sd	Store the value of $\overline{s1}$ into sd
IMCSR	0000000	110	IMCSR $s1$, $s2$, sd	Circular shift of $s2$ by $s1$ bits to the right, store result in sd
IMSR	0000000	111	IMSR $s1$, $s2$, sd	Shift of $s2$ by $s1$ bits to the right, store result in sd
IMCSL	1000000	000	IMCSL $s1$, $s2$, sd	Circular shift of $s2$ by $s1$ bits to the left, store result in sd
IMSL	1000000	001	IMSL $s1$, $s2$, sd	Shift of $s2$ by $s1$ bits to the left, store result in sd

(b)

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

To support IMCRYPTO operations, we introduce eight RV32I I-type [see Table I (a)] and ten RV32I R-type [see Table I(b)] customized instructions in the RISC-V instruction set architecture (ISA). Two RV32I I-type instructions are used to load/store from/to IMCRYPTO's CEM (discussed in Section III-A) to/from registers in the RISC-V core. The rest of the I-type instructions are used to trigger each step in AES encryption/decryption. After decoding these customized I-type instructions, the RISC-V-based controller generates three main inputs to each block of IMCRYPTO—one enable (EN) bit, one memory address (ADDR), and one piece of data (four registers of 32 bits each). When IMCRYPTO finishes executing an instruction, it sends back a "DONE" signal to the RISC-V-based controller, which then fetches the next instruction.

The ten RV32I R-type customized instructions are used to trigger general-purpose bitwise and arithmetic in-memory operations, such as IMAND, IMOR, IMXOR, and IMADD, inside the CEM in IMCRYPTO. These instructions are used to realize the arithmetic and logic operation inside the memory (IMXOR is also used to achieve the AddRoundKey step of AES). As the in-memory customized R-type instructions are decoded, one enable bit (EN), two source memory addresses, and one destination memory address (ADDR) are generated. The CEM operates on the two source memory addresses specified by instructions, performs the corresponding in-memory operation, and stores the result in the destination memory address. The aforementioned control signals (for both RV32I I-type and R-type instructions) are showed in Fig. 4(a).

We use the opcodes 0000111 and 1000111 to differentiate the regular RISC-V instructions from our customized instructions I-type and R-type. Furthermore, as IMCRYPTO's instructions are executed inside the CEM, the arithmetic logic unit (ALU) and memories inside the RISC-V processor can be simultaneously used to execute regular RISC-V instructions running in different threads. For example, IMCRYPTO can use one RISC-V thread to receive data. By employing this multithreaded technique, the RISC-V core acts as a controller for our IMCRYPTO accelerator, while it is also available to be used as a general-purpose processor.

By supporting different in-memory operations via the customized RISC-V R-type instructions, IMCRYPTO can achieve various modes of operation for AES without any hardware modifications. For instance, the CTR mode of AES leverages a counter value, which can be stored inside the CEM of IMCRYPTO. The counter can then be implemented with the use of IMADD operations. As another example, the CBC mode of AES for processing streaming data can be achieved by an additional IMXOR operation between the output values of two consecutive AES 128-bit blocks.

In addition to the different modes of AES, IMCRYPTO can also support other block ciphers and hash algorithms, such as Blowfish, CAST-128, SHA-256, and MD5, which has similar computation operations and complexity as AES. By implementing these algorithms in IMCRYPTO, we can take advantage of high parallelism, high throughput, low data transfer time, and low energy consumption offered by IMC.

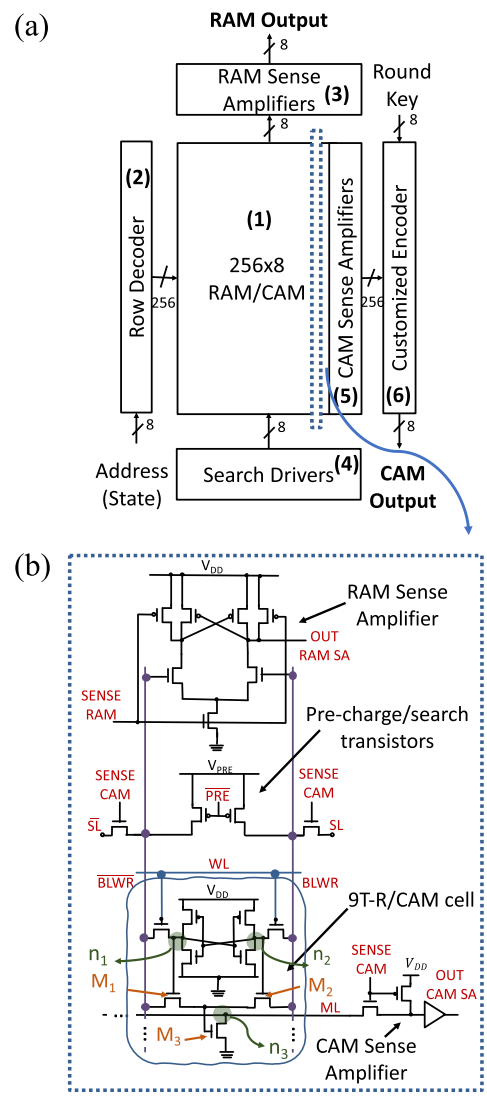


Fig. 5. (a) High-level view of the RA/CAM array. (b) Schematic view of one column of the array (with one RA/CAM cell included).

In Section V, we present a use case for an IMCRYPTO-based implementation of the SHA-256 hashing algorithm.

IV. RA/CAM Design

Here, we introduce the RA/CAM design in IMCRYPTO. Fig. 5(a) provides a high-level view of the components of the RA/CAM design: 1) the 256 × 8 RA/CAM array; 2) the row decoder; 3) the RAM SAs; 4) the search drivers; 5) the CAM SAs; and 6) the customized encoder.

Our RA/CAM design is based on CMOS static RAM (SRAM) memory cells. While 6T-SRAM cells can be employed for the design of RAM modules (e.g., [24]), our RA/CAM cell design employs nine transistors (similar to the cell proposed in [19]). Although Li [19] emphasizes the usage of 9T-SRAM cells in the CAM setting only, in IMCRYPTO, we introduce peripheral circuits (i.e., a row decoder and RAM SAs) that enables the array to work as both a RAM or a CAM (hence, a RA/CAM array is comprised of 9T-RA/CAM cells). The CAM functionality of the RA/CAM array enables both

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

the SubBytes and InvSubBytes steps to be performed with a common structure for encryption and decryption.

The schematic view of one column of the RA/CAM array is shown in Fig. 5(b). Although we only show one 9T-RA/CAM cell and the per-column peripherals in the figure, our array design has 256 9T-RA/CAM cells connected to a common column. These memory cells share one RAM SA, and two precharge and two search transistors. Furthermore, the array has eight columns. Memory cells in the same row and different columns share one CAM SA. RA/CAM has two access modes. In the following, we detail the operation of these two modes.

A. RAM Mode

Our 256×8 RA/CAM array performs writes and reads in the RAM mode. The writes and reads with the 9T-RA/CAM are similar to the write and read operations with a 6T-SRAM (e.g., [24]). At the end of a write operation, the nodes n_1 and n_2 store opposite logic states. The transistors M_1, M_2, and M_3 are not used in the RAM mode.

B. CAM Mode

The CAM mode enables a parallel search for any byte value in the range 0×00–0xFF among the 256 bytes stored in the 256×8 RA/CAM array. The search drivers in Fig. 5(a) distribute the byte pattern to the eight columns of the array (through SL and $\overline{\text{SL}}$). Before a search operation, a 0-V voltage pulse is applied to BLWR and $\overline{\text{BLWR}}$ to ensure that any residual voltage at the node n_3 is discharged [see Fig. 5(b)]. To initiate a search, we assert SENSE CAM $= V_{\text{DD}}$, which activates the search transistors connected to SL and $\overline{\text{SL}}$ and precharges the matchline ML to V_{DD}.

As mentioned in Section IV-A, after a memory cell is written, the nodes n_1 and n_2 store opposite logic states, which implies that either the transistor M_1 or the transistor M_2 is turned on during search (i.e., M_1 and M_2 are never turned on simultaneously). At this point, if the logic state stored in n_1 (n_2) is equal to the logic state applied to the SL ($\overline{\text{SL}}$), the node n_3 remains discharged, and the M_3 transistor remains off. In this scenario, the ML is not discharged, producing a match at the output of the CAM SA (OUT CAM SA $= V_{\text{DD}}$).

Alternatively, if the state stored in n_1 (n_2) is the opposite of the state applied to the SL ($\overline{\text{SL}}$), there is a path from SL ($\overline{\text{SL}}$) to n_3 (through either M_1 or M_2), which enables the node n_3 to charge up and the transistor M_3 to turn on. In this scenario, the ML is discharged, producing a mismatch at the output of the CAM SA (i.e., OUT CAM SA $= 0V$).

C. AES Mapping

IMCRYPTO offers advantages from a VLSI and architecture standpoints when computing AES. First, IMCRYPTO has enough flexibility to implement multiple AES modes and even other security algorithms through customized instructions on an RISC-V processor (see Section V for an example). Second, CTs can be stored inside IMCRYPTO's CEM, an encrypted LLC that enables fast access by applications. The execution of AES with IMCRYPTO is facilitated by our hardware-software codesign approach, which consists of mapping of AES steps to the proposed IMC fabric, as outlined in the next paragraphs.

IMCRYPTO realizes AddRoundKey with its CEM. The structure of the CEM, which is based on numerous tiled arrays, enables parallel computations on a wide length of data (i.e., the 128 bits of one AES block). XOR operations occur between the round key and the PT, which are temporarily stored in CEM's memory. These operations are performed with the help of customized SAs. The results of AddRoundKey are placed in latches at the output of the SAs.

IMCRYPTO realizes the byte permutations of ShiftRows/InvShiftRows with bidirectional shifters. The shift amount can be set freely between 0 and 4 bytes, in both directions, which allows flexibility for implementing AES and other cryptographic algorithms. Like with AddRoundKey, the results of ShiftRows/InvShiftRows are placed in latches at the output of the bidirectional bit shifters.

IMCRYPTO realizes SubBytes+MixColumns and InvSubBytes+InvMixColumns through a combined step with its LUT fabric. Details of the step combination approach and its implementation in hardware are provided in Section IV-D. The advantages of combining steps in AES are reduction of delay, with a good area compromise. Like with the other steps, the results of SubBytes+MixColumns and InvSubBytes+InvMixColumns are placed in latches at the output of the LUT fabric.

D. Step Combination in IMCRYPTO

To maximally exploit IMC and reduce computation cost, we leverage a step combination technique in IMCRYPTO. Specifically, we store the values of $1 \times \text{sbox}(a_{i,j})$ in the four RA/CAM arrays and the values of $2 \times \text{sbox}(a_{i,j})$ and $3 \times \text{sbox}(a_{i,j})$ in the eight RAM arrays of one LUT fabric module (instead of simply storing $\text{sbox}(a_{i,j})$). These values are needed by the matrix multiplication operation in the MixColumns step; hence, storing them directly in the RA/CAM arrays enables us to easily combine the SubByte and MixColumns steps of AES encryption. The InvSubBytes, AddRoundKey, and InvMixColumns steps of AES decryption are also executed in a combined fashion with the modification of peripheral circuits (i.e., the customized encoder) of the RA/CAM array.

1) Encryption: Fig. 6(a) illustrates the execution of the SubBytes and MixColumns in an LUT fabric module with the RAM arrays and RA/CAM working in the RAM mode. Bytes $a_{i,j}$ in the same column of the input state array (i.e., bytes that have the same j index) are given as memory addresses to the row decoders of the eight RAM arrays and four RA/CAM arrays in a transposed fashion. In the example of Fig. 6(a), we use $j = 0$. A precomputed byte in each array is read out by the RAM SA at the given address. Once the bytes are read out from the RA/CAM arrays, they are XORed by the XOR tree to produce the result of the matrix multiplication in the MixColumns step for the column j in the output state array.

2) Decryption: To avoid unnecessary power and delay overheads, IMCRYPTO does not require RA/CAM arrays to be reprogrammed when the accelerator switches its functionality from encryption to decryption. Therefore, during decryption, the four RA/CAM arrays of one LUT fabric module contain the same values of encryption (i.e., $1 \times \text{sbox}(a_{i,j})$). The

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

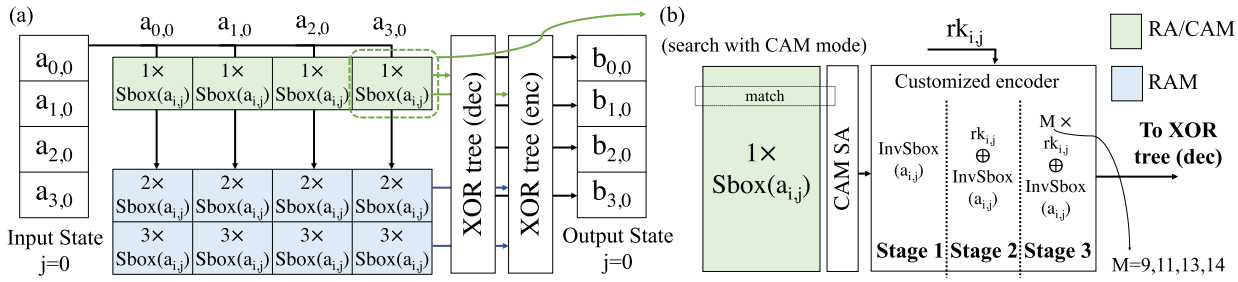


Fig. 6. Combined (a) SubBytes and MixColumns steps for AES encryption, and (b) InvSubBytes, AddRoundKey, and InvMixColumns steps for AES decryption.

Algorithm 1 SHA-256: Message Padding

Input: Initial message w
Output: Padded message w

for $i = 16,\ i < 64,\ i++$ **do**
  $s0 := \sigma 0(w[i-15])$;
  $s1 := \sigma 1(w[i-2])$
  $w[i] := (w[i-16] + s0 + w[i-7] + s1$
return w

Algorithm 2 SHA-256: Main Hashing Function

Input: Padded message w, initial hashes $A - H$, constant k
Output: Resulting Hashes A,B,C,D,E,F,G,H

for $i = 16,\ i < 64,\ i++$ **do**
  $s0 := \Sigma 0(A)$;
  $s1 := \Sigma 1(E)$;
  $ch := Choice(E, F, G)$;
  $maj := Majority(A, B, C)$;
  $temp1 := H + s1 + ch + k[i] + w[i]$;
  $temp2 := s0 + maj$;
  $H := G$;
  $G := F$;
  $F := E$;
  $E := D + temp1$;
  $D := C$;
  $C := B$;
  $B := A$;
  $A := temp1 + temp2$;
return A,B,C,D,E,F,G,H

RAM arrays in the LUT fabric module are not used in AES decryption and can be turned off.

Fig. 6(b) depicts IMCRYPTO's approach for combining steps in AES decryption. IMCRYPTO performs InvSubBytes in the periphery of the RA/CAM array operating in the CAM mode (i.e., with customized encoders). Customized encoders allow the execution of InvSubBytes to be combined with AddRoundKey and InvMixColumns to enable efficient AES decryption. We divide the execution of combined steps in AES decryption with IMCRYPTO into stages. Stage 1 corresponds to the operations of InvSubBytes and leverages the invertible nature of the Sbox function to perform the SubBytes and InvSubBytes with distinct modes of a RA/CAM array. Stage 2 produces the result of AddRoundKey. Finally, Stage 3 computes the multiplication of InvMixColumns.

In the example of Fig. 6(b), bytes $a_{i,j}$ with the same j index are given as search data to the search drivers of the four RA/CAM arrays in a transposed fashion (similar to encryption). Although we illustrate only the operations in the last RA/CAM arrays in our LUT fabric module (i.e., where the search data is $a_{3,0}$), identical operations occur simultaneously in the other columns of arrays ($a_{0,0}$, $a_{1,0}$, and $a_{2,0}$). In Stage 1 of our approach, we search for the byte $a_{3,0}$ in the RA/CAM arrays. The CAM SAs detect a single match for each array (i.e., while one row of the RA/CAM array produces a match, 255 rows produce mismatches). Since the RA/CAM array stores $1 \times \text{sbox}(a_{i,j})$, the row which returned a match is encoded into a byte as the equivalent result of the InvSubBytes step.

In Stage 2, the customized encoder simply performs an XOR operation between each InvSubBytes result and $rk_{0,0}$ (i.e., a byte of the round key) in order to produce the result of the AddRoundKey step. In Stage 3, the 8-bit result of the AddRoundKey step is multiplied by 9, 11, 13, and 14. This multiplication can be done with combinational logic inside the customized encoder that maps 1-byte value into another.

Similar to encryption, during decryption, the four 8-bit outputs of the customized encoder at Stage 3 are XORed with the other outputs of the first row of RA/CAM arrays in the LUT fabric module (i.e., those where search data inputs were $a_{1,0}$, $a_{2,0}$, and $a_{3,0}$) with the XOR tree. The final result, which corresponds to the output of the InvSubBytes step, is forwarded to the next block in the AES decryption datapath.

V. Use Case of IMCRYPTO Beyond AES

The distinct security needs by various applications, in addition to the development of new algorithms for handling emerging attacks, make it highly desirable for crypto engines to be flexible. Furthermore, configurable crypto engines that can "guarantee interoperability between countries and institutions" are highly sought-after IP blocks [11]. In IMCRYPTO, the use of an RISC-V coprocessor allows for basic in-memory operations to be executed in an arbitrary order, which enables support for different modes of AES and other

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

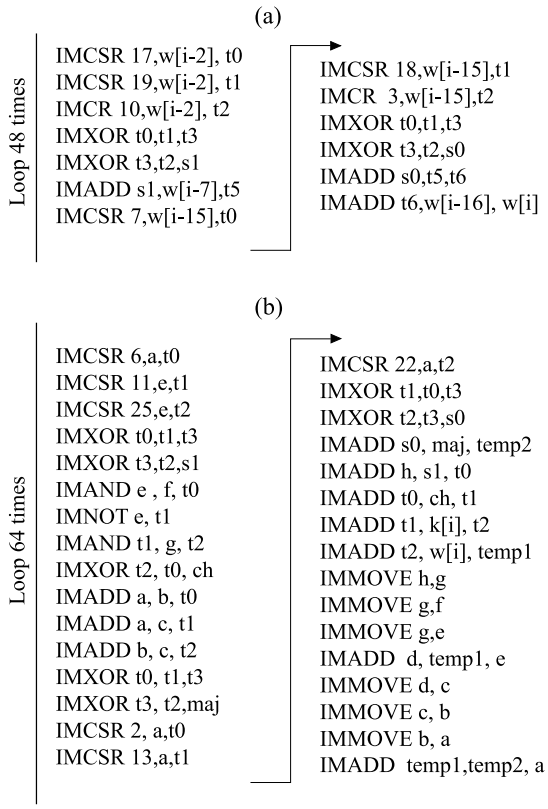


Fig. 7. Translation of SHA-256 (a) message padding (see Algorithm 1) and (b) main hashing function (see Algorithm 2) into customized instructions.

encryption/decryption algorithms beyond AES (e.g., hashing functions and other symmetric block ciphers). In this section, we provide an example for implementing SHA-256 on our IMCRYPTO fabric.

SHA-256 is a member of SHA-2—a group of cryptographic hash functions designed by the National Security Agency (NSA). Widely known for its high security and speed, SHA-256 is used in the block chain industry and encryption communication protocols. The SHA-256 algorithm can be divided into two parts: *message padding* (see Algorithm 1) and *main hashing function* (see Algorithm 2). While the former extends a 16-bit word message to a 64-bit word message, the latter generates fixed-sized outputs (i.e., hashes). Message padding uses the σ_0 and σ_1 functions for padding (lines 2 and 3 in Algorithm 1). σ_0 and σ_1 are implemented with bitwise "right-rotate" and "right-shift" operations, as defined in the following:

$$\sigma_0(A) = (A \ggg 7) \oplus (A \ggg 18) \oplus (A \gg 3)$$
$$\sigma_1(A) = (A \ggg 17) \oplus (A \ggg 19) \oplus (A \gg 10).$$

IMCRYPTO uses the customized instructions IMCSL, IMCSR, IMSR, IMXOR, and IMADD to perform message padding [see Fig. 7(a)]. For instance, to implement rotations inside IMCRYPTO, the data are accessed through their memory addresses. Then, these data are read through the customized SA of the CEM and sent to hardware blocks that implement in-memory bit shifters. Fixed shift amounts are used with the bit shifters in order to slide the data bits to specific positions as required by the rotation operation.

TABLE II
ACCELERATORS CONSIDERED IN THE EVALUATION

Design	Type	Technology Node	Supported Algorithms	Reference
A	ASIC	CMOS 45 nm	AES encryption/ decryption	[4]
B	ASIC	CMOS 45 nm	AES encryption+ Most symmetric ciphers	[5]
C	IMC	CMOS 40 nm	AES encryption, Keccak, finite field multiply	[6]
D	IMC	PCM 65 nm	AES encryption/ decryption	[7]
E	IMC	CMOS 45 nm	AES encryption/ decryption*	IMCRYPTO

Notes: *IMCRYPTO can potentially support other symmetric ciphers by manipulating its custom instructions/reordering the steps executed in the IMC architecture.

The main hashing function (see Algorithm 2) is executed after message padding and produces the final output (i.e., a set of 8, fixed-sized hashes). The main hashing function in SHA-256 takes 64 rounds to complete its execution. It is comprised of two functions—"majority" and "choice"

$$\text{Majority}(A, B, C) = A \cdot B \oplus A \cdot C \oplus B \cdot C$$
$$\text{Choice}(A, B, C) := A \cdot B \oplus \overline{A} \cdot C.$$

Each of these functions takes three inputs A, B, and C, and performs computation as follows: 1) the "majority" function outputs "0" ("1") when half or more of its inputs are "0" ("1") and 2) the "choice" function uses input A as a selector bit, i.e., when $A =$ "1″("0″), the function outputs the value of 'B'('C'). The main hashing function uses the Σ_0 and Σ_1 functions to hash the data, which can be implemented with bitwise "right-rotate" operations

$$\Sigma_0(A) = (A \ggg 2) \oplus (A \ggg 13) \oplus (A \ggg 22)$$
$$\Sigma_1(A) = (A \ggg 6) \oplus (A \ggg 11) \oplus (A \ggg 25).$$

Besides these functions, 64 constants +8 initial hashes (A–H) are needed. IMCRYPTO prestores these constants and the initial hashes inside the CEM and uses the customized instructions, IMCSR, IMXOR, IMAND, IMOR, IMNOT, IMADD, and IMMOVE [see Table I (b)], to perform the operations in the main hashing function.

VI. EXPERIMENTAL EVALUATION

Here, we compare figures of merit (FoMs) (i.e., delay, throughput, area, power, throughput per area, and ADPP) of IMCRYPTO with accelerators for AES encryption/decryption from previous works [4]–[6].

A. Experimental Setup

In our evaluation (and comparison), we implement AES-128 encryption (ECB mode). The implementation of the IMCRYPTO instructions in the RISC-V-based controller and their sequencing for AES-128 ECB mode is done in Verilog and evaluated through register transfer level (RTL)

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

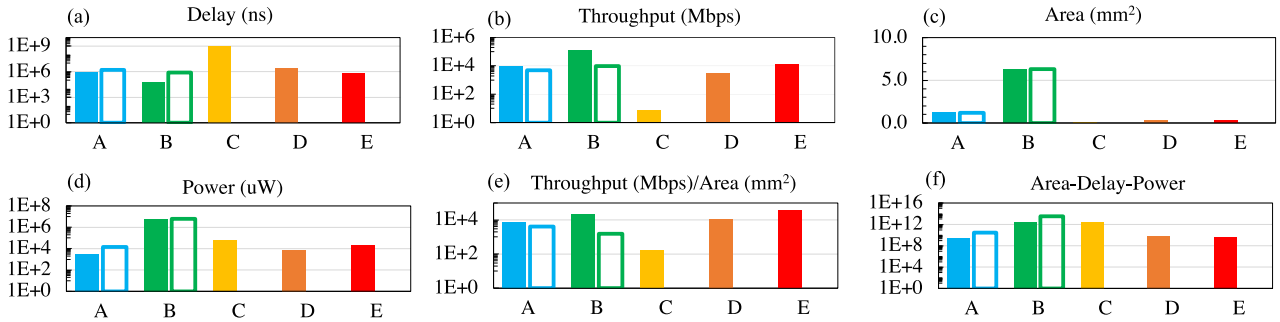


Fig. 8. (a) Delay, (b) throughput, (c) area, (d) power, (e) throughput per area, and (f) ADPP of AES-128 encryption. We compare the AES accelerators listed in Table II. Filled (hollow) bars do not (do) include the latency and energy overhead of data transfers to/from a memory unit for the ASIC-based accelerators (designs A and B).

simulations. At the circuit level, we use the DESTINY simulator [25][1] to measure area, latency, and energy for a 1-MB compute-enabled SRAM with a two-mat organization and a subarray size of 128 × 128. We have modified DESTINY to support the customized SAs and local write buffers of [15]. The compute-enabled SRAM is based on a 45-nm CMOS predictive technology model (PTM) [27]. We choose the 45-nm technology node as most designs in our comparison use this node. To estimate the area overhead introduced by compute circuits in the SRAM, we subtract the area of a baseline SRAM of 1 MB based on the same technology node from the area of the compute-enabled RAM. We use DESTINY to simulate the baseline SRAM, which is a regular SRAM memory without customized peripherals. The results indicate an area overhead of 6.9% for the SRAM due to the addition of the compute circuits, which is compatible with the 6.0% area overhead reported in [15].

The area, latency, and energy of the bidirectional shifter block are measured through synthesis using the Cadence Encounter RTL Compiler v14.10 with the NanGate 45-nm open-cell library [28]. Finally, the 256 × 8 RA/CAM arrays in IMCRYPTO's LUT fabric [block 4 of Fig. 4(a)] are simulated using HSPICE version O-2018.09-1 with the PTM for 45-nm CMOS [27] and $V_{DD} = 1$ V. We measure the latency and energy for the different types of memory accesses, i.e., reads/writes in RAM mode and searches in the CAM mode. In order to estimate the area of the 256 × 8 RA/CAM array, we employ the OpenRAM memory compiler [29], in which we specify the 6T-SRAM and 9T-RA/CAM cells, as well as the peripherals of our design. The area of the IMCRYPTO fabric (which we compare with the area of other accelerators in Section VI-C) includes the area overhead of compute-enabled SRAM, the area of the bidirectional shifter, and the area of all circuits in the LUT fabric.

After Verilog synthesis and SPICE simulations, we carefully check the log files (i.e., simulation reports) for implementation issues, such as inferred latches, outputs without external loads, and nets with multiple drivers. Furthermore, in our evaluation, we test multiple input values (i.e., for the encryption of 1 byte, all possible cases, 0 to 255, are simulated) to make sure we get good coverage in our tests.

[1]DESTINY is an open-source tool to simulate 2-D and 3-D memory arrays, which utilizes the 2-D modeling framework of NVSim [26] for SRAM and nonvolatile memories based on emerging technologies.

B. Frequency of Operation

When performing one round of AES-128 encryption for a 128-bit block of PT, the latency of the IMCRYPTO blocks labeled 1–4 in Fig. 4(a) is given as follows.

1) *Block 1 (CEM):* It executes the XOR operation for the AddRoundKey step of encryption/decryption inside the 1-MB CEM in under 870 ps.
2) *Block 2 (Bidirectional Shifter):* It executes the byte permutation operation for the ShiftRows/InvShiftRows steps of encryption/decryption in under 100 ps.
3) *Block 3 (LUT Fabric):* It executes the lookups and XOR operations needed in Galois field matrix multiplications for the combined SubBytes+Mixcolumns/InvSubBytes+InvMixcolumns steps in under 780 ps.
4) *Block 4 (RISC-V-Based Controller):* It adds no more than 100 ps to each encryption/decryption step to fetch and decode the customized instructions that are executed with IMCRYPTO's circuits in blocks 1–3.

Considering the latencies outlined above, the worst case scenario latency with IMCRYPTO is the latency of the CEM (870 ps for in-memory operation+100 ps for instruction decoding and fetching). Hence, we have chosen the operation frequency of 1 GHz, which is adopted throughout our evaluation. Furthermore, we take advantage of IMCRYPTO's modular structure to pipeline instructions through block 1 (mats 0 and 1), block 2, and block 3 of the architecture in order to achieve higher throughput.

C. Quantitative Analysis With Different FoMs

Table II summarizes previous work on accelerators for AES along with our proposed design IMCRYPTO (design E). Designs A and B are ASIC accelerators for AES based on a 45-nm CMOS technology node. Designs C–E are based on the IMC concept and consist of CEM arrays integrated with customized circuitry that compute at the memory periphery. While designs C and E are based on 40- and 45-nm CMOS technologies, respectively, design D is based on a 65-nm PCM technology. Designs B and C support other algorithms/functions besides AES encryption. Support for AES decryption

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

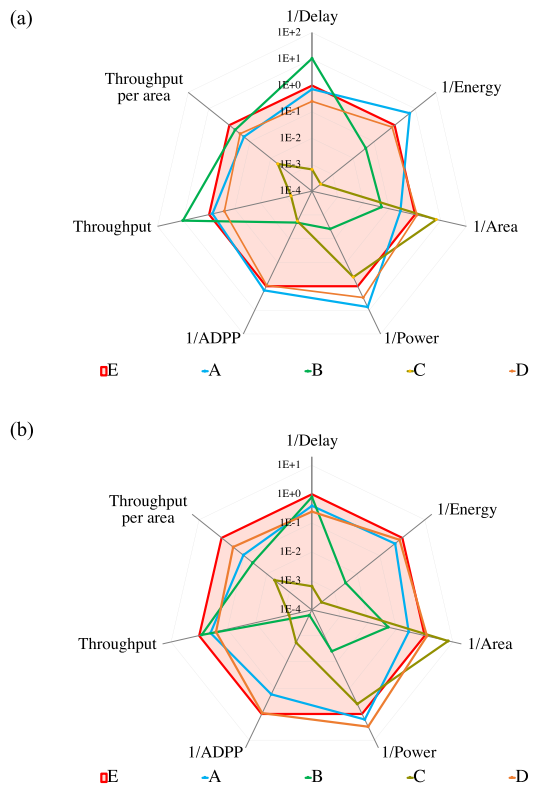


Fig. 9. Comparison between designs A–E for seven different FoMs (a) without and (b) with the overhead of data transfers.

is not discussed in [5] and [6]. AES decryption is possible with designs A, D, and E.

Figs. 8(a)–(f) and 9(a) and (b) present and compare different FoMs for designs A–E for AES-128 encryption. While Fig. 8(a)–(f) presents the raw data for all FoM in separate bar graphs, Fig. 9(a) and (b) enables us to collectively compare the different designs in terms of all FoM evaluated. This comprehensive presentation of several FoMs favors the study of throughput–area tradeoffs, energy efficiency, and so on. Different use cases may favor different FoM, which may serve to distinct design objectives depending on the application. For instance, if IMCRYPTO is used for accelerating AES encryption/decryption in a resource-constrained environment (i.e., an embedded system), then a balance between throughput and area with low power consumption is essential. In this regard, the presented FoM should be analyzed collectively, so the benefit can be better understood under different circumstances.

The scores for the FoM presented are delay in ns (measures the time for encrypting 1 MB of data), throughput in Mb/s (measures the volume of data that can be encrypted in 1 s), area in mm^2 (corresponds to the silicon area occupied by the accelerator), and power in μW (corresponds to total static and dynamic power of the design during encryption). In addition, we also present derived scores. such as throughput per area, defined as the ratio between the throughput of the accelerator and the silicon area occupied by the design. Another derived metric is the ADPP, which is defined as the multiplication between the design area, the delay for encrypting 1 MB of data, and the power of the design. FoM scores (i.e., improvements) are calculated.

Two sets of data are considered. The filled bars in Fig. 8(a)–(f) [lines in Fig. 9(a)] *do not* include the latency and energy of data transfers to/from a memory unit for the two ASIC-based accelerators (designs A and B). The FoM comparison in Fig. 9 are normalized to IMCRYPTO. This normalization is computed as the inverse of the ratio between delay, energy, area, power, and ADPP metrics for design A, B, C, or D, and the same metrics for design E (IMCRYPTO). Note that, in the case of throughput and throughput per area, the ratio does not require inversion for better visualization of the benefits when put close to the other metrics.

Note that designs C—E are based on the IMC paradigm, which allows computation to be performed inside (or near) the memory without the need for data transfers. The second set of data [hollow bars in Fig. 8(a)–(f)/lines in Fig. 9(b)] adds 744.49-μs latency (30.39-μJ energy) from memory transfers to the latency (energy) of designs A and B. This additional latency/energy is spent on transferring 1 MB of PT from a 1-MB SRAM to each ASIC accelerator and writing back the resulting 1 MB of CT to the same 1-MB SRAM. DESTINY [25] is used to estimate the latency and energy overheads of such data transfers.

We first consider delay and throughput when data transfers are *not* accounted for in designs A and B (i.e., the ASIC-based accelerators). In this scenario, design B has the shortest delay to encrypt 1 MB of data and the highest throughput among all designs evaluated [see Fig. 8(a) and (b)]. IMCRYPTO has the highest throughput per area [see Fig. 8(e)] among all designs in our study. Compared to designs A–D, design E enables minimum improvements in throughput per area of $\sim$5.0$\times$, $\sim$ 1.9$\times$, $\sim$ 223.1$\times$, and $\sim$ 3.3$\times$, respectively.

When we include the overhead of data transfers for the ASIC-type accelerators in our analysis, design E has the shortest delay and highest throughput among all designs. Namely, compared to design B, design E is 1.2$\times$ better with respect to both delay and throughput. Importantly, fast and high-throughput AES-128 encryption in IMCRYPTO (design E) comes with an improvement of 19.7$\times$ in terms of area [see Fig. 8(c)] and 257.7$\times$ in terms of power [see Fig. 8(d)] compared to design B. Finally, when we account for data transfers, design E outperforms design B by a factor of 5.7$\times$ in terms of ADPP [see Fig. 8(f)].

Note that, in terms of the ADPP metric, design A would be the best-in-class design when data transfers for ASIC-based designs *are excluded* from the evaluation. In the setup where data transfers are included in the total delay and energy results, the second-best design in terms of ADPP is design D, an architecture also based on IMC. Unlike design E (which is based on CMOS), design D employs PCM and bank-level parallelism in its CEM. Design E slightly improves the ADPP by a factor of 1.2$\times$ compared to design D, which is associated with the two designs occupying roughly the same area, with design E having a 4.0$\times$ shorter delay and higher power consumption (0.3$\times$). Some of the benefits of design D are derived from the use of PCM. In fact, IMCRYPTO can also benefit from alternative memory technologies that we discuss next.

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

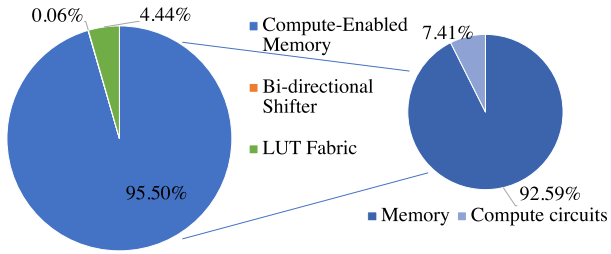


Fig. 10. Area breakdown of the IMCRYPTO fabric based on a 1-MB CEM. Memory components dominate the overall area of the proposed accelerator.

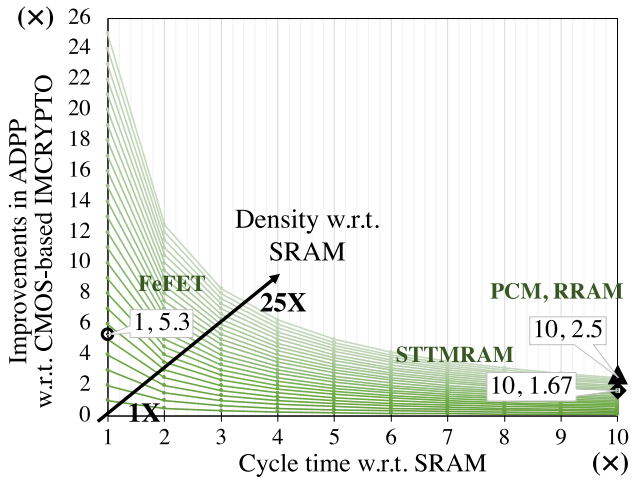


Fig. 11. Projected improvements in ADPP with respect to CMOS-based IMCRYPTO for memories based on emerging technologies. STT-MRAM, PCM, and RRAM density and cycle time data points are from [8]. FeFET density and cycle time are from [30]. The pair of numbers for each technology indicate the cycle time with respect to SRAM and improvements in ADPP with respect to CMOS-based IMCRYPTO.

D. FoM Projection Based on Emerging Technologies

Per Fig. 10, the CEM and the LUT fabric modules of IMCRYPTO are responsible for >99.9% of the total area of our proposed AES encryption/decryption IMC accelerator. As discussed in Section III, these two blocks of IMCRYPTO are built with CMOS 6T-SRAM and 9T-RA/CAM cells. Here, we investigate whether the use of memories based on emerging technologies, such as PCM, RRAM, STT-MRAM, and FeFETs, to replace CMOS-based memories could result in additional benefits to our proposed IMCRYPTO approach.

Per [8], [9], and [30], emerging technologies could potentially improve the density of memories by factors of $1\times-25\times$. Furthermore, the read power of memories based on most emerging technologies may be comparable to that of CMOS SRAM with static power savings due to nonvolatility [9]. Despite these advantages, one of the downsides of emerging technologies is their long read and write latencies (up to $10\times$ of CMOS SRAM), which may increase the clock cycle time and reduce the frequency of operation of IMC accelerators, such as IMCRYPTO. To analyze the impact of emerging technologies on the ADPP of IMCRYPTO, we study the tradeoff between longer cycle times (thus lower operating frequencies) and density improvements enabled by emerging technologies with respect to CMOS SRAM (see Fig. 11).

In our projection, we consider the read power of emerging technologies to be on par with that of CMOS SRAM [9]. The green curves in the Fig. 11 represent different memory densities ($1\times$–$25\times$ compared to SRAM). Given cycle times that are up to $10\times$ longer than those of conventional SRAM (x-axis), improvements in ADPP with respect to CMOS-based IMCRYPTO (y-axis) are captured. From the data points depicted in Fig. 11 (from [8] and [30]), we project that current state-of-the-art memory technologies improve the ADPP of IMCRYPTO by $1.7\times$–$5.3\times$. For instance, by implementing a PCM-based IMCRYPTO, we project $2.5\times$ benefits in ADPP relative to a CMOS-based IMCRYPTO (design E in Table II). As CMOS-based IMCRYPTO's ADPP slightly outperforms PCM-based design D by a factor of $1.2\times$, when we compare the two designs implemented with the same technology, we expect design E to outperform design D in terms of ADPP by a factor of $3.0\times$.

Despite the projections, we acknowledge that considerable challenges from the system/architecture perspective exist in employing emerging technologies for the design of memories and IMC-based accelerators [31]. For instance, technologies may be at different stages of maturity and not be as easily scalable to smaller nodes as in CMOS technology. Nevertheless, as memory technologies continue to evolve, opportunities emerge with the study of new materials, the possibility of stacked memory, and so on. The impact of these advancements on the performance of IMC for security applications is worthwhile to pursue in further investigations.

E. Side-Channel Attack Analysis

The communication between a traditional CPU and a coprocessor (hardware accelerator) for AES encryption/decryption may be vulnerable to time and power side-channel attacks. For the former, we note that, although faster, it is a well-known fact that table-based implementations of Sbox are vulnerable to cache timing attacks in software implementations that run on a traditional CPU architecture [16]. Nevertheless, table-based implementations of Sbox are well-suited for FPGA- or ASIC-based accelerator designs, which does not employ caches in their construction [16]. IMCRYPTO employs an LUT fabric comprised of static RA/CAM and 6T-SRAM arrays that store precomputed values for the combined SubBytes and MixColumns steps [see Fig. 4(b)]. These precomputed values are not fetched from a cache structure, and once stored in the LUT fabric, the tables remain unchanged through the execution of the whole AES. Importantly, the access time of RA/CAM (and 6T-SRAM) arrays in the LUT fabric [see Fig. 4(b)] does not show any data dependency pattern, which makes IMCRYPTO resilient to cache timing attacks (such as FPGA- and ASIC-based accelerator designs).

We next discuss the tolerance of IMCRYPTO to correlation power analysis (CPA) attacks. To this end, our ongoing work is investigating the resistance of LUT fabric in IMCRYPTO against power analysis attacks that target the correlation between the total power consumption of the hardware and the secret keys [32]. The LUT fabric of IMCRYPTO can potentially provide resistance to power analysis attacks due to

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.

the symmetric design of RA/CAM cells that store complementary bits in their cross-coupled inverters. In our preliminary experiments, we have observed that the difference between power consumption from reading different values from the RA/CAM (i.e., loading 1 or loading 0) is extremely small, which makes it hard to find the correlation between the input of the RA/CAM array and power consumption. Due to the variety of existing power side-channel attacks, and the complexity of analysis and experiments needed to verify the resistance of the whole IMCRYPTO architecture against power analysis attacks, we leave reporting this part of the findings to future work.

VII. Conclusion and Future Work

We propose IMCRYPTO, an IMC fabric for accelerating AES encryption and decryption that enables high throughput while avoiding large area/power overheads. IMCRYPTO employs a novel approach that combines multiple RAM and RA/CAM arrays with customized encoders in the design of a modular LUT fabric. With the proposed LUT fabric, the (Inv)SubBytes and (Inv)MixColumns steps from AES encryption (decryption) can be combined and performed in one shot in a highly parallel fashion. Furthermore, integration with RISC-V allows IMCRYPTO to have the flexibility of doing different modes of operation and opens the door for the implementation of other ciphers that require operations that are similar to those of AES. Our future work will evaluate the execution of different AES modes and key lengths with IMCRYPTO and will extend our customized instructions to enable the execution of other symmetric-key ciphers. Furthermore, we will exploit different levels of control granularity by replacing a subset of customized instructions in IMCRYPTO with AES "macroinstructions." Finally, we will investigate the resilience of the LUT fabric in IMCRYPTO to CPA attacks.

References

[1] J. Daemen and V. Rijmen, "AES proposal: Rijndael," Nat. Inst. Standards Technol., U.S. Dept. Commerce, Washington, DC, USA, Tech. Rep. FIPS PUB 197, 2001.

[2] K. Stevens and O. A. Mohamed, "Single-chip FPGA implementation of a pipelined, memory-based AES rijndael encryption design," in *Proc. Can. Conf. Electr. Comput. Eng.*, May 2005, pp. 1296–1299.

[3] R. Chaves, G. Kuzmanov, S. Vassiliadis, and L. Sousa, "Reconfigurable memory based AES co-processor," in *Proc. 20th IEEE Int. Parallel Distrib. Process. Symp.*, Apr. 2006, p. 8.

[4] R. Ueno *et al.*, "High throughput/gate AES hardware architectures based on datapath compression," *IEEE Trans. Comput.*, vol. 69, no. 4, pp. 534–548, Apr. 2020.

[5] G. Sayilar and D. Chiou, "Cryptoraptor: High throughput reconfigurable cryptographic processor," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2014, pp. 155–161.

[6] Y. Zhang, L. Xu, Q. Dong, J. Wang, D. Blaauw, and D. Sylvester, "Recryptor: A reconfigurable cryptographic cortex-M0 processor with in-memory and near-memory computing for IoT security," *IEEE J. Solid-State Circuits*, vol. 53, no. 4, pp. 995–1005, Apr. 2018.

[7] M. Xie, S. Li, A. O. Glova, J. Hu, and Y. Xie, "Securing emerging nonvolatile main memory with fast and energy-efficient AES in-memory implementation," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 11, pp. 2443–2455, Nov. 2018.

[8] S. Yu and P.-Y. Chen, "Emerging memory technologies: Recent trends and prospects," *IEEE Solid State Circuits Mag.*, vol. 8, no. 2, pp. 43–56, Spring 2016.

[9] G. Sun, J. Zhao, M. Poremba, C. Xu, and Y. Xie, "Memory that never forgets: Emerging nonvolatile memory and the implication for architecture design," *Nat. Sci. Rev.*, vol. 5, no. 4, pp. 577–592, Jul. 2018.

[10] S. Angizi *et al.*, "Accelerating deep neural networks in processing-in-memory platforms: Analog or digital approach?" in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Jul. 2019, pp. 197–202.

[11] L. Bossuet, M. Grand, L. Gaspar, V. Fischer, and G. Gogniat, "Architectures of flexible symmetric key crypto engines—A survey: From hardware coprocessor to multi-crypto-processor system on chip," *ACM Comput. Surv.*, vol. 45, no. 4, p. 41, Aug. 2013, doi: 10.1145/2501654.2501655.

[12] A. Sasongko, I. M. N. Kumara, A. Wicaksana, F. Rousseau, and O. Müller, "Hardware context switch-based cryptographic accelerator for handling multiple streams," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 14, no. 3, pp. 1–25, Sep. 2021, doi: 10.1145/3460941.

[13] B. Ege, E. B. Kavun, and T. Yalçin, "Memory encryption for smart cards," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.* Springer, 2011, pp. 199–216.

[14] W. Shi, H.-H.-S. Lee, M. Ghosh, C. Lu, and A. Boldyreva, "High efficiency counter mode security architecture via prediction and precomputation," in *Proc. 32nd Int. Symp. Comput. Archit. (ISCA)*, Jun. 2005, pp. 14–24.

[15] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das, "Compute caches," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2017, pp. 481–492.

[16] E. Tromer, D. A. Osvik, and A. Shamir, "Efficient cache attacks on AES, and countermeasures," *J. Cryptol.*, vol. 23, no. 1, pp. 37–71, 2010.

[17] M. J. Dworkin, "Recommendation for block cipher modes of operation: Methods and techniques," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Special Rep. 800-38a 2001 edition, 2001.

[18] S. Almuhammadi and I. Al-Hejri, "A comparative analysis of AES common modes of operation," in *Proc. IEEE 30th Can. Conf. Electr. Comput. Eng. (CCECE)*, Apr. 2017, pp. 1–4.

[19] H. Li, "A new CAM based S1-box look-up table in AES," in *Proc. ISCAS*, vol. 5, 2005, pp. 4634–4636.

[20] S. K. Mathew *et al.*, "53 Gbps native $GF(2^4)^2$ composite-field AES-encrypt/decrypt accelerator for content-protection in 45 nm high-performance microprocessors," *IEEE J. Solid-State Circuits*, vol. 46, no. 4, pp. 767–776, Apr. 2011.

[21] A. Gupta and M. Jaiswal, "An enhanced AES algorithm using cascading method on 400 bits key size used in enhancing the safety of next generation Internet of Things (IoT)," in *Proc. Int. Conf. Comput., Commun. Autom. (ICCCA)*, May 2017, pp. 422–427.

[22] J. Wolkerstorfer, E. Oswald, and M. Lamberger, "An ASIC implementation of the AES SBoxes," in *Topics in Cryptology–(CT-RSA)*, B. Preneel, Ed. Berlin, Germany: Springer, 2002, pp. 67–78.

[23] D. Reis, M. T. Niemier, and X. S. Hu, "A computing-in-memory engine for searching on homomorphically encrypted data," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 5, pp. 123–131, 2019.

[24] X. Si *et al.*, "A dual-split 6T SRAM-based computing-in-memory unit-macro with fully parallel product-sum operation for binarized DNN edge processors," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 11, pp. 4172–4185, Nov. 2019.

[25] M. Poremba, S. Mittal, D. Li, J. S. Vetter, and Y. Xie, "DESTINY: A tool for modeling emerging 3D NVM and eDRAM caches," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, San Jose, CA, USA, Mar. 2015, pp. 1543–1546.

[26] X. Dong, C. Xu, Y. Xie, and N. P. Jouppi, "NVSim: A circuit-level performance, energy, and area model for emerging nonvolatile memory," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 7, pp. 994–1007, Jul. 2012.

[27] Y. Cao, *Predictive Technology Model for Robust Nanoelectronic Design*. Boston, MA, USA: Springer, 2011.

[28] J. Knudsen, "Nangate 45 nm open cell library," CDNLive, EMEA, Amsterdam, The Netherlands, Tech. Rep., 2008.

[29] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, "OpenRAM: An open-source memory compiler," in *Proc. 35th Int. Conf. Comput.-Aided Design*, Nov. 2016, pp. 1–6.

[30] D. Reis *et al.*, "Design and analysis of an ultra-dense, low-leakage, and fast FeFET-based random access memory array," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 5, pp. 103–112, 2019.

[31] O. Mutlu, "Memory scaling: A systems architecture perspective," in *Proc. 5th IEEE Int. Memory Workshop*, May 2013, pp. 21–25.

[32] O. Lo, W. J. Buchanan, and D. Carson, "Power analysis attacks on the AES-128 S-box using differential power analysis (DPA) and correlation power analysis (CPA)," *J. Cyber Secur. Technol.*, vol. 1, no. 2, pp. 88–107, Apr. 2017.

Authorized licensed use limited to: ULAKBIM UASL - Galatasaray Universitesi. Downloaded on November 06,2022 at 06:51:59 UTC from IEEE Xplore. Restrictions apply.